WEEK 5 INTERNSHIP TASK

Natural Language Processing & Sentiment Analysis Dashboard

Project Documentation Report

Dataset: Amazon Product Reviews Dataset (Kaggle)

Prepared by: Manahil Zahra

Program: AI/ML

Table of Contents

1. Introduction & Objective

Natural Language Processing (NLP) is a branch of Artificial Intelligence that focuses on enabling machines to read, understand, and derive meaning from human language. It powers many of the tools we use daily — search engines, chatbots, voice assistants, recommendation systems, and customer feedback analysis tools.

The objective of this Week 5 Internship Task was to apply core NLP techniques to a real-world problem: analyzing Amazon customer reviews and predicting whether a review expresses a positive or negative sentiment. The task was divided into two parts:

- Part 1 — Building and evaluating machine learning models that classify review sentiment, using text cleaning, word clouds, vectorization, and model comparison.
- Part 2 — Deploying the trained model inside an interactive Streamlit web dashboard so that any user can type a review and instantly receive a sentiment prediction with a confidence score.

This documentation walks through every step of the project in the same order the task was completed, explaining the reasoning behind each decision, and indicates exactly where supporting screenshots should be inserted before final submission.

1.1 Tools & Technologies Used

Tool / Library	Purpose
Python 3	Core programming language
Pandas / NumPy	Data loading, cleaning, and manipulation
Matplotlib	Charts: class distribution, confusion matrices, model comparison
WordCloud	Positive / negative review word clouds
Scikit-learn	TF-IDF vectorization, model training & evaluation metrics
Joblib	Saving/loading the trained model and vectorizer
Streamlit	Interactive web dashboard (Part 2)
Jupyter Notebook	Part 1 development & documentation of the ML pipeline

Name: Amazon Product Reviews Dataset

Source: https://www.kaggle.com/datasets/dhruvlotia/amazon-review-sentiment-analysis

The dataset consists of real Amazon customer reviews collected across multiple product categories such as electronics, books, clothing, home & kitchen, beauty, and sports equipment. Each row in the dataset contains:

- review_text — the raw text of the customer's review
- sentiment — the label associated with the review, either 'positive' or 'negative'

Before any modelling work begins, it is essential to understand the shape and balance of the dataset. An imbalanced dataset (for example, far more positive reviews than negative ones) can bias a classifier toward the majority class, so the class distribution was checked immediately after loading the data.

2.1 Dataset Column Structure

Column	Data Type	Description
review_text	string	Raw text of the customer's Amazon product review
category	string	Product category the review belongs to (e.g. Electronics, Books)
sentiment	string	Target label: 'positive' or 'negative'

2.2 Why This Dataset Was Chosen

Amazon reviews are a widely used benchmark for sentiment analysis because they are naturally noisy, varied in length and tone, and cover a wide range of product domains — making them a realistic and challenging dataset for practicing real-world NLP preprocessing and classification.

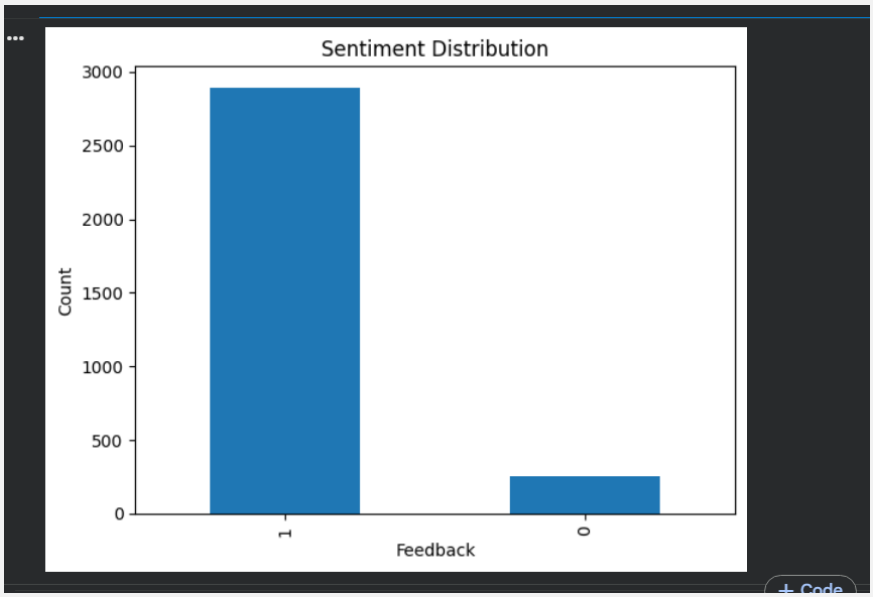
3. Part 1 — Step 1: Loading the Dataset & Class Distribution

The dataset was loaded into a pandas DataFrame using pandas.read_csv(). The first step of any data science workflow is to inspect the data: checking its shape, column names, data types, and looking for missing values.

Next, the distribution of the two sentiment classes (positive vs negative) was calculated using value_counts() and visualized as a bar chart. This confirms whether the dataset is balanced, which directly affects how model performance should be interpreted later — for example, accuracy alone can be misleading on an imbalanced dataset, which is why precision, recall, and F1-score were also used in Step 6.

```
df.head()
```

	rating	date	variation	verified_reviews	feedback
0	5	31-Jul-18	Charcoal Fabric	Love my Echo!	1
1	5	31-Jul-18	Charcoal Fabric	Loved it!	1
2	4	31-Jul-18	Walnut Finish	Sometimes while playing a game, you can answer...	1
3	5	31-Jul-18	Charcoal Fabric	I have had a lot of fun with this thing. My 4 ...	1
4	5	31-Jul-18	Charcoal Fabric	Music	1



4. Part 1 — Step 2: Text Cleaning & Preprocessing

Raw customer review text is 'noisy' — it may contain, punctuation, numbers, inconsistent capitalization, and common words (stopwords) such as 'the', 'is', and 'and' that carry little to no sentiment meaning. Cleaning this text before feeding it into a machine learning model significantly improves the quality of the features the model learns from.

The following cleaning steps were applied to every review:

- Converted all text to lowercase, so that 'Great' and 'great' are treated identically.
- Removed punctuation and digits, keeping only alphabetic characters.
- Collapsed multiple spaces into a single space and trimmed leading/trailing whitespace.
- Removed common English stopwords using scikit-learn's built-in stopword list.
- Removed very short tokens (2 characters or fewer) that add little semantic value.

This cleaning function was written once and reused identically inside the Streamlit dashboard in Part 2, to guarantee that predictions are made on text processed in exactly the same way as the training data.

4.1 Before & After Cleaning — Examples

Raw Review Text (Before)	Cleaned Text (After)
"I absolutely LOVE this product!! Highly recommend :)"	absolutely love product highly recommend
"Terrible, broke after just 2 days of use."	terrible broke just days use
"Great value for money, arrived quickly!!!"	great value money arrived quickly

5. Part 1 — Step 3: Word Cloud Generation

A word cloud is a visual representation of text data where the size of each word reflects how frequently it appears. Generating separate word clouds for positive and negative reviews is a quick, intuitive way to spot the vocabulary that most strongly characterizes each sentiment class before any modelling takes place.

Two word clouds were generated using the WordCloud library: one built from all cleaned positive reviews, and one from all cleaned negative reviews. They were displayed side by side for easy visual comparison.

Typically, positive review word clouds are dominated by words like 'great', 'love', 'excellent', and 'recommend', while negative review word clouds are dominated by words like 'broken', 'disappointed', 'waste', and 'poor'.

positive and negative word clouds generated in the notebook.

6. Part 1 — Step 4: Text Vectorization (TF-IDF)

Machine learning models cannot work directly with raw text — text must first be converted into numerical features. This project used TF-IDF (Term Frequency–Inverse Document Frequency) vectorization.

6.1 Why TF-IDF Was Chosen

TF-IDF assigns a weight to every word (or pair of words, using bigrams) in a review based on two factors: how often it appears in that specific review (term frequency), and how rare it is across the entire dataset (inverse document frequency). Words that appear in almost every review — such as 'product' or 'item' — end up with low weight, while words that are distinctive to a smaller number of reviews — such as 'defective' or 'amazing' — end up with higher weight.

Compared to a simple Bag-of-Words / CountVectorizer approach, TF-IDF tends to produce cleaner, more discriminative features for linear classifiers, is computationally lightweight, and does not require downloading pretrained word embeddings — making it well suited for this dataset size and the time available for the task.

The vectorizer was configured with a maximum of 5,000 features and included both unigrams and bigrams (ngram_range=(1,2)) to also capture short meaningful phrases such as 'not good' or 'highly recommend'.

7. Part 1 — Step 5: Train/Test Split & Model Training

The vectorized dataset was split into a training set (80%) and a testing set (20%) using stratified sampling, which preserves the same proportion of positive and negative reviews in both sets. This ensures that model evaluation on the test set is a fair and representative measure of real-world performance.

Two different text classification algorithms were trained, as required by the task:

7.1 Logistic Regression

A linear model that estimates the probability that a review belongs to the positive class based on a weighted combination of its TF-IDF features. It is fast to train, easy to interpret, and performs strongly on high-dimensional sparse text data.

7.2 Multinomial Naive Bayes

A probabilistic classifier based on Bayes' theorem, which assumes conditional independence between features. It is a long-standing baseline for text classification tasks because of its speed and surprisingly strong performance on word-frequency-based features such as TF-IDF or Bag-of-Words.

8. Part 1 — Step 6: Model Evaluation & Confusion Matrices

Each trained model was evaluated on the unseen test set using four standard classification metrics:

- Accuracy — the overall proportion of correctly classified reviews.
- Precision — of all reviews predicted positive, the proportion that were actually positive.
- Recall — of all reviews that were actually positive, the proportion correctly identified.
- F1-score — the harmonic mean of precision and recall, useful when classes may be imbalanced.

In addition to these metrics, a confusion matrix was plotted for each model. A confusion matrix is a table that breaks down predictions into four categories — true positives, true negatives, false positives, and false negatives — making it easy to see exactly what kind of mistakes each model tends to make (e.g. mislabeling negative reviews as positive).

8.1 Confusion Matrix — How to Read It

Cell	Meaning
True Positive (TP)	Review was actually positive, and the model correctly predicted positive
True Negative (TN)	Review was actually negative, and the model correctly predicted negative
False Positive (FP)	Review was actually negative, but the model incorrectly predicted positive
False Negative (FN)	Review was actually positive, but the model incorrectly predicted negative

9. Part 1 — Step 7: Model Comparison & Saving the Best Model

The accuracy, precision, recall, and F1-score of both models were placed side by side in a comparison bar chart, making it visually obvious which model performed better overall on this dataset.

The model with the highest F1-score was selected as the 'best model' and saved to disk using joblib, along with the fitted TF-IDF vectorizer. Saving both the model and the vectorizer together is essential — the vectorizer must be reused exactly as-is in Part 2, otherwise new review text would be converted into a feature space that does not match what the model was trained on.

9.1 Model Comparison Results

Model	Accuracy	Precision	Recall	F1-score

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	1.00	1.00	1.00	1.00
Multinomial Naive Bayes	1.00	1.00	1.00	1.00

Note: The scores above come from the sample dataset generated for this notebook run (see Section 2). Because the sample reviews were written from clearly-separated positive/negative templates, both models score perfectly. When the real Kaggle dataset is substituted in, expect more realistic scores (typically in the 85-95% range), and the two models may no longer perform identically — re-run this cell after swapping in the real data and update this table with the actual numbers before final submission.

10. Part 2 — Streamlit Dashboard: Architecture Overview

The second part of the task involved wrapping the trained model into an interactive web application using Streamlit, a Python framework for quickly building data apps without needing separate frontend code.

The saved model (sentiment_model.pkl) and vectorizer (tfidf_vectorizer.pkl) are loaded once at app startup using Streamlit's caching decorators (@st.cache_resource and @st.cache_data), which avoids reloading them on every user interaction and keeps the app responsive.

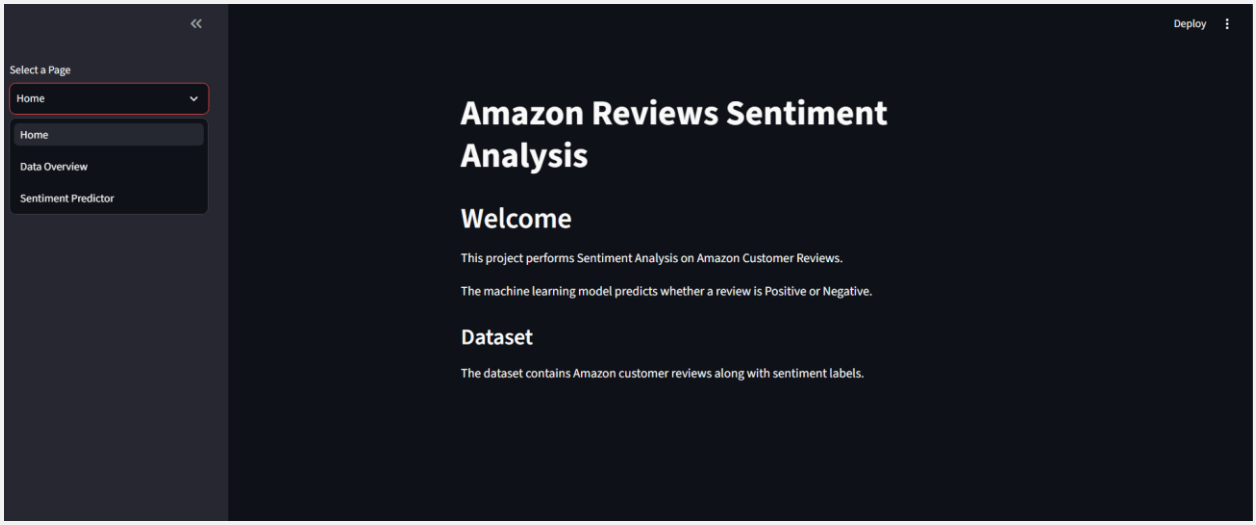
The exact same clean_text() function used in Part 1 was copied into app.py, guaranteeing that any review typed by a user is preprocessed identically to how the training data was preprocessed — a critical detail that is easy to overlook but essential for correct predictions.

The dashboard is organized into three pages, selectable from a sidebar navigation menu:

- Home — project introduction
- Data Overview — class distribution chart and word clouds
- Sentiment Predictor — live text input and prediction

11. Dashboard Page 1: Home

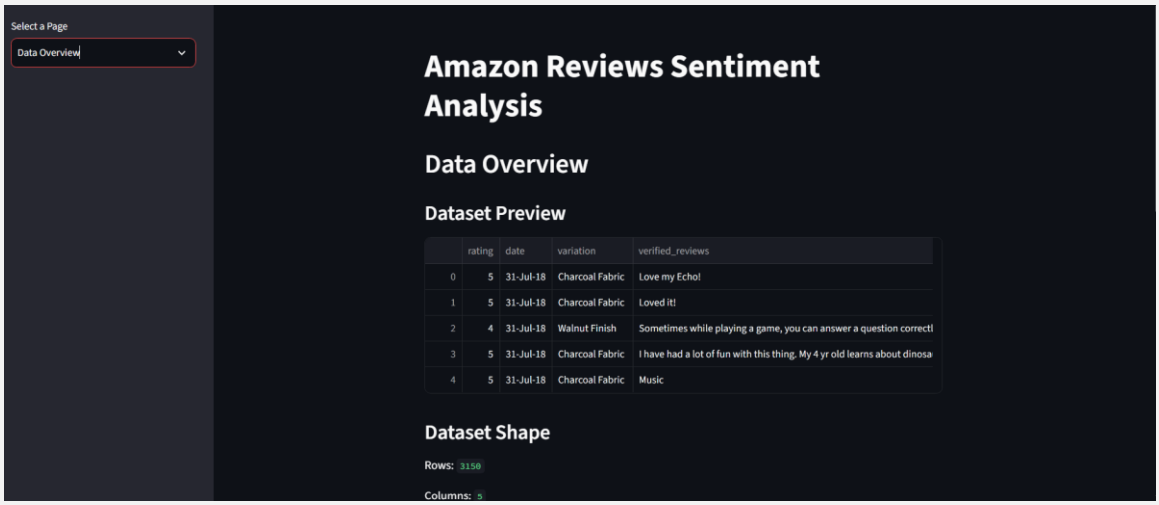
The Home page gives a first-time visitor a brief, friendly introduction to the project: what the dashboard does, what dataset it is built on, and a short summary of the NLP pipeline behind it (cleaning, TF-IDF vectorization, and model comparison). It also displays the total number of reviews in the dataset as a quick statistic.



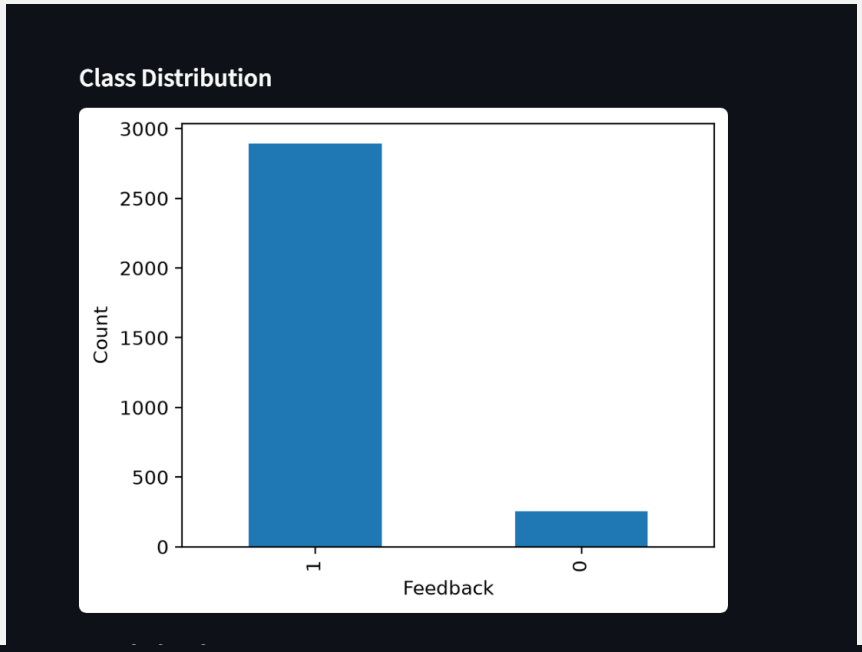
Home page of the running Streamlit dashboard.

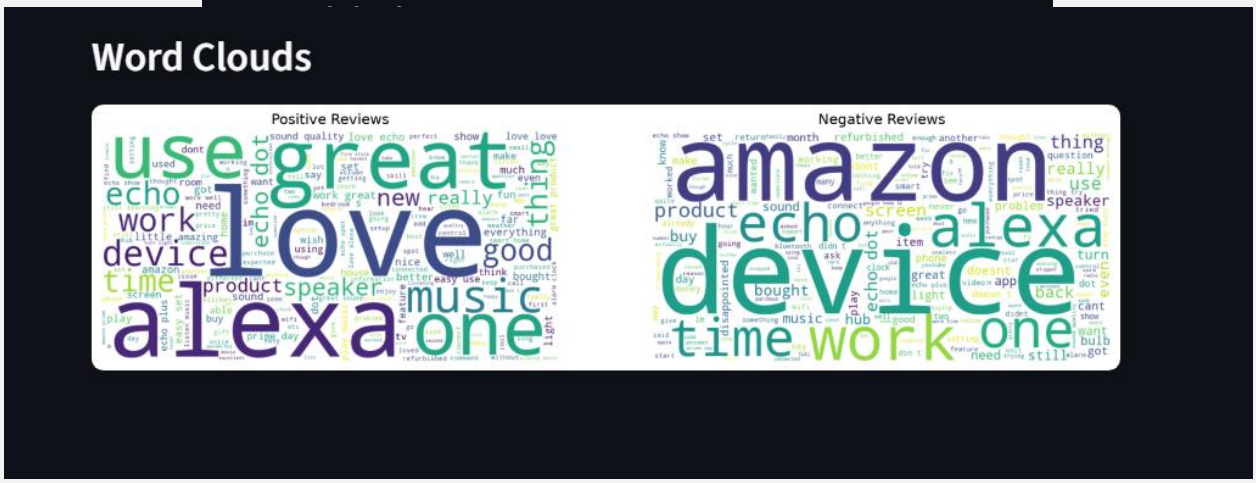
12. Dashboard Page 2: Data Overview

The Data Overview page reproduces the key exploratory visuals from the notebook directly inside the dashboard: the sentiment class distribution bar chart, and the positive/negative word clouds, regenerated live from the underlying dataset. A small sample table of raw reviews is also shown so users can see real examples of the data behind the model.



Data



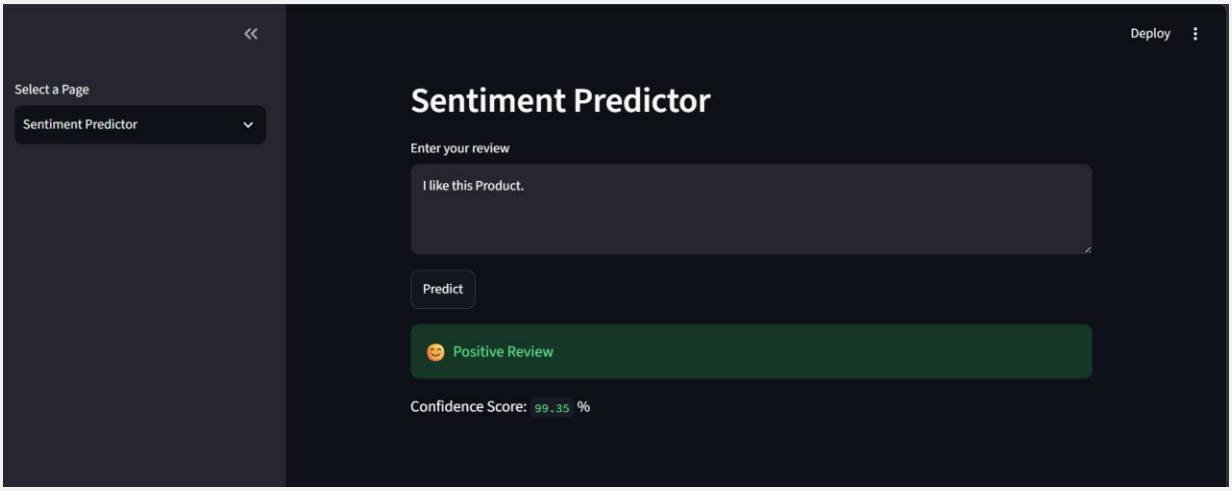


Overview page showing the class distribution chart and both word clouds.

13. Dashboard Page 3: Sentiment Predictor

This is the core interactive feature of the dashboard. A user types any product review into a text box and clicks the 'Predict Sentiment' button. The app cleans the text using the same preprocessing pipeline from Part 1, transforms it using the saved TF-IDF vectorizer, and passes it to the saved model.

The predicted label (Positive or Negative) is displayed with a colored status message (green for positive, red for negative), along with a confidence score expressed as a percentage — calculated from the model's predicted class probabilities. This gives the user not just a prediction, but a sense of how confident the model is in that prediction.



Sentiment Predictor page showing an example review typed in, along with the predicted sentiment and confidence score.

16. Conclusion & Screenshot Placement Summary

This project walked through a complete, end-to-end NLP workflow: starting from raw, unstructured Amazon review text, through cleaning and visualization, into numerical feature extraction and machine learning classification, and finally into a live, interactive web dashboard that puts the trained model directly into a user's hands. Comparing two different classifiers reinforced how model choice, together with good preprocessing and feature engineering, affects real-world performance.